IaC DRIFT DETECTION & AI-DRIVEN REMEDIATION

By Neeraj Pandey

DevOps Conf 2026

[DAY 1: SOLVED]

```
$ terraform apply

Apply complete!
Resources: 3 added

[OK] Infrastructure provisioned
[OK] All systems operational
```

AI CODE GENERATION WORKS

[DAY 2+: REALITY]

```
! DRIFT DETECTED

! sg-0abc123 modified
! Port 22 → 0.0.0.0/0
! CIS violation
! 4 resources at risk
! 19-hour exposure
```

OPERATIONS PROBLEM UNSOLVED

IaC DRIFT DETECTION & AI-DRIVEN REMEDIATION

Bridging AI-Generated Infrastructure & Operational Reality

WHO & WHERE WE'RE GOING

$ whoami

CTO @ Lyntcube & Vivid Climate

Vivid: Carbon/Water Accounting + DMRV
Lyntcube: AI Operators for Real Estate

Building compliance infrastructure at scale.

$ whoami --verbose

This talk is for:
- [x] Platform engineers
- [x] DevOps building AI tools
- [x] SREs handling drift
- [x] Security evaluating auto-remediation
- [x] Leaders assessing AI readiness

You will leave with:
- → Working code (agents/*.py)
- → Production prompts (2,600 words)
- → OPA policies (3-tier Rego)
- → Pre-production checklist

$ cat agenda.txt

[1] Drift Problem
→ AI solved Day 1 code gen, but Day 2+ operations still broken

[2] Detection Fails
→ 19-hour incident: 3 tools, 3 teams, 7 alerts, 0 coordination

[3] Unified Pipeline
→ Graph + Causal AI achieves 85% alert reduction

[4] AI Agents: RCA + Fix
→ Detective + Surgeon + Checker = 75% → 95% accuracy

[5] Governance
→ OPA 3-tier policies, machine identity, never runs apply

[6] Failure Modes
→ Circuit breaker, 900 → 30 alerts/day, production lessons

[7] Takeaways + Q&A
→ Copy-paste templates, <$300/mo, implementation guide

[WARNING] Two real production failures dissected.
Not hypotheticals. Real postmortems.

DevOps Conf 2026

BEAUTIFUL LIE: DAY 1 SOLVED

$ ./modern_pipeline.sh

[1] PROMPT

[OK] "Create encrypted S3 bucket with versioning"

[2] LLM GENERATES

[OK] 47 lines HCL · Correct AWS provider syntax

[3] SECURITY SCAN

[OK] Checkov + tfsec · 0 violations

[4] PR REVIEW

[OK] Approved · Merged to main

[5] CI/CD

[OK] terraform plan + apply · 3 resources

[6] LIVE

[OK] All systems operational

**"Generation problem solved.
Operations problem fragmented, rule-based, and require manual investigation"**

```
$ diff day1.txt operations.txt

[SOLVED] Code generation
[SOLVED] Security validation
[SOLVED] Deployment automation

[MISSING] Drift detection
[MISSING] Root cause analysis
[MISSING] Auto-remediation
[MISSING] Operational context

Generation: 100% solved
Operations: 0% started
```


DevOps Conf 2026

DRIFT HAPPENS: INEVITABLE

$ ./reality_check.sh

[DRIFT: The Inevitable Truth]

Three Root Causes:

[1] Manual Hotfix
"I'll revert it later" — they never do
→ Console clicks bypass Git

[2] Out-of-Band Automation
Lambda / SaaS modifying resources
→ Separate from IaC pipeline

[3] Incident Response
SRE changes infra at 2am
→ PR deferred forever

[INSIGHT]: Drift is not a failure of IaC.
It's a property of humans operating
dynamic systems under pressure.

[DECLARED STATE]

```
aws_security_group.api_sg {
  ingress {
    cidr_blocks = ["10.0.0.0/8"]
    description = "Internal only"
  }
}
```

[ACTUAL STATE]

```
aws_security_group.api_sg {
  ingress {
    cidr_blocks = ["0.0.0.0/0"]  ← DRIFT!
    description = ""
  }
}
```

$ cat drift_stats.txt

PRODUCTION REALITY:

Avg time to first drift: 18 hours
Drift events per week: 12-15
Source breakdown:
- Manual: 60%
- Automated: 25%
- Incidents: 15%

Detection without tooling: Days to weeks
Impact: Silent accumulation

DevOps Conf 2026

DRIFT = MULTI-DOMAIN INCIDENT

$ analyze --impact=full

INFRASTRUCTURE

t3.micro → m5.4xlarge

$5,200 unexpected bill

SECURITY

Port 22 → 0.0.0.0/0

CIS violation · active exposure

NETWORK

Route table modified

45-min service disruption

COMPLIANCE

RDS encryption disabled

Audit failure · regulatory fine

[KEY INSIGHT]

A single console click on a security group can simultaneously be:

→ An infrastructure drift
→ A security incident
→ A network policy violation
→ A compliance finding

Your current tooling sees these as 3 separate unrelated alerts.
Across 3 teams. On 3 different timelines. No shared context.

DevOps Conf 2026

THREE TOOLS · ZERO COORDINATION

$ ps aux | grep detection

[INFRAOPS]

terraform plan
driftctl
Atlantis

Sees: attr diffs
Action: Create Terraform PR

T+4min:
sg-0abc123
CIDR drifted

Ticket: **INFRA-8472**
Queue: Platform Team

[SECOPS]

Prisma Cloud
AWS Config
SecurityHub

Sees: policy violations
Action: Flag for remediation

T+3hr:
Port 22 open
to internet

Ticket: **SEC-2109**
Queue: Security Team

[NETOPS]

Batfish
NetBox
NAPALM

Sees: route/ACL diffs
Action: Check network impact

T+9hr:
(nothing—it's
an SG, not route)

Ticket: **None**
Queue: Not routed

[THE GAP]

No shared resource model
No blast radius analysis
No causal chain
3 tickets · 3 queues · 3 SLAs

Same root cause. Three fragments. Zero picture.

DevOps Conf 2026

INCIDENT NOBODY CAUGHT IN TIME

$ cat postmortem.txt

```
COMPOSITE INCIDENT POSTMORTEM
Service: api.platform.example.com
Exposure Window: 19 hours
──────────────────────────────────────────

T+0:00  Developer opens sg-0abc123 to 0.0.0.0/0:22
        Console change. Fast. "Just for quick debug."
        No ticket. No PR. Will revert it. Later.

T+0:04  Terraform state polling detects drift
        → InfraOps ticket: "sg-0abc123 CIDR changed"
        → Priority: MEDIUM (just a CIDR diff, no context)
        → SLA: 4 hours. Queued.

T+3:44  CSPM scan completes (runs every 3 hours)
        → SecOps alert: "CIS 5.2 — SSH open to internet"
        → Priority: HIGH
        → NEW SecOps ticket, fresh investigation started
        → No link to the InfraOps ticket. No shared context.

T+4:00  InfraOps closes their ticket: "No service impact"
        (they saw only the diff — not the security implication)

T+9:15  NetOps morning routing review
        VPC flow anomaly on instances behind sg-0abc123
        Third ticket opened.

──────────────────────────────────────────
EXPOSURE: 19 hours · 2 EC2 instances · Port 22 world-open
TEAMS INVOLVED: 3 · TICKETS CREATED: 3 · CORRELATED: 0
──────────────────────────────────────────

The InfraOps engineer wasn't wrong.
The SecOps engineer wasn't wrong.
The system was designed to produce this outcome.
```


DevOps Conf 2026

GRAPH + CAUSAL AI

```
$ ./analyze_blast_radius.py
```

[PROPERTY GRAPH MODEL]

```
[VPC]
  ├── [Subnet: private-1]
  │       └── [SG: sg-0abc123]  ◄── DRIFT NODE
  │                ├──[SECURES]──► [EC2: app-prod-1]
  │                │                    ├──[CONNECTS_TO]──► [RDS: db-prod]
  │                │                    └──[MOUNTS]──────► [EFS: fs-01]
  │                └──[SECURES]──► [EC2: app-prod-2]
  │                                     └──[CONNECTS_TO]──► [RDS: db-prod]

OLD MODEL:  "sg-0abc123 CIDR changed"
            → 1 resource, 1 alert

GRAPH MODEL: "sg-0abc123 CIDR changed"
            → Direct risk:    2 EC2 instances
            → Transitive:     RDS, EFS (via EC2)
            → Blast radius:   4 downstream resources
```

[CAUSAL AI]

Correlation ≠ Causation

```
GRAPH tells you: WHAT is affected
CAUSAL AI tells you: WHY it happened


Correlation:  CPU spike coincided with
deployment
Causation:    Did deployment cause spike, or
              did spike trigger auto-remediation
              that looked like deployment?


PC Algorithm on event time-series:
  [deploy] → [sg_change] → [cpu_spike]
  (not: [cpu_spike] → [sg_change])


Practical output:
  "Root cause: sg_change (T+0:04)
   caused by: deploy event (T-0:02)
   NOT by: the cpu alert (T+0:19)"
```

```
[IMPLEMENTATION]

Graph DB: NetworkX (dev) · Neo4j (production)
Causal: CausalLearn (PyWhy) · PC Algorithm
Risk Score: resource_criticality × (1.0 + blast_radius × 0.05)  ── Capped at 1.3× (30% increase max)

[EXAMPLE]
Database with 0 connections: 1.0 × 1.0 = 1.0 (critical by nature)
S3 bucket with 10 connections: 0.6 × 1.5 = 0.9 (becomes critical through usage)
```


DevOps Conf 2026

PC ALGORITHM & CAUSALLEARN

[PC ALGORITHM]

```
Learns causal structure
from observations (Peter Clark)

[Observations]
      ↓
┌──────────────┐
│ 1. SKELETON  │ Find all
│  Chi² test   │ relationships
└──────────────┘
      ↓
 A ─── B ─── C
 │           │
 └── D ──────┘
      ↓
┌──────────────┐
│2. V-STRUCT   │ Detect
│  A → B ← C   │ colliders
└──────────────┘
      ↓
 A → B ← C
 ↓       ↓
 D ────── E
      ↓
┌──────────────┐
│ 3. ORIENT    │ Direct
│ Meek rules   │ all edges
└──────────────┘
      ↓
[Causal DAG]
```

[CAUSALLEARN]

```
PyWhy library for causal
discovery from data

from causallearn.search \
  .ConstraintBased.PC import pc

# Build observation matrix
data = build_matrix(
  drift_events,
  window=3600
)

# Run PC algorithm
cg = pc(
  data,
  alpha=0.05,
  indep_test=fisherz
)

# Extract causal edges
for i, j in cg.G.edges():
  src = RESOURCES[i]
  tgt = RESOURCES[j]
  dag.add_edge(src, tgt)
```

[NETWORKX DAG]

```
Graph operations: blast
radius + root cause

import networkx as nx

dag = nx.DiGraph()

# Add directed edges
dag.add_edge(
  "sg-0abc123",
  "i-001",
  weight=0.9
)

# Calculate blast radius
downstream = list(
  nx.descendants(
    dag,
    "sg-0abc123"
  )
)

# Find root causes
root = [
  n for n in dag.nodes()
  if dag.in_degree(n)==0
]
```

[OBSERVATION MATRIX: Input to PC]

```
Time-series drift events converted to binary matrix
Each row = 1-hour window | Each col = resource type

Time     SG   EC2  RDS  EFS  Lambda  VPC
──────────────────────────────────────────
14:00     1    0    0    0     0     0
15:00     0    1    1    0     0     0
16:00     0    0    1    1     0     0
17:00     0    0    0    0     1     0

data = np.array([[1,0,0,0,0,0], [0,1,1,0,0,0], ...])
cg = pc(data, alpha=0.05, indep_test=fisherz)
```

[CAUSAL DAG OUTPUT]

```
PC discovers cause → effect relationships
from timing + statistical tests

14:00  [SG drift]
         ↓ CAUSES (0.9)
15:00  [EC2 drift] → [RDS drift]
         ↓ CAUSES (0.85)
16:00  [EFS mount timeout]

ROOT CAUSE: sg-0abc123 (in_degree=0)
BLAST RADIUS: 3 downstream resources
CORRELATION: 4 alerts → 1 root cause
```

Libraries: causal-learn (PC) · NetworkX (DAG) · NumPy (matrix)

UNIFIED PIPELINE: 5 LAYERS

*** THE ANCHOR SLIDE ***

```
[LAYER 1]: DATA SOURCES  (existing tools stay in place)

[Terraform State]  [CloudTrail Events]  [OPA Engine]  [Net Baselines]
Poll every 15m      EventBridge stream   Continuous     Diff every 30m

[LAYER 2]: NORMALIZE → DriftEvent Schema (the contract)

[LAYER 3]: CORRELATION ENGINE  (Neo4j / AWS Neptune)

  [Graph Model]  ──▶  [Blast Radius]  ──▶  [Causal Inference]
  [Deduplication: same resource + 15-min window = 1 event]
  [Severity = MAX across correlated group]
  [RESULT: 7 raw alerts → 1 correlated incident · -85% noise]

[LAYER 4]: AI AGENT ORCHESTRATOR

  [Detective: RCA + Context]  ──▶  [Surgeon: Patch + PR]

[LAYER 5]: GITOPS + OPA GATE + HUMAN APPROVAL

  [OPA Policy] → [Team Routing] → [PR Review] → [terraform apply]
```

CRITICAL CONSTRAINTS:

① No existing tools are replaced
② DriftEvent schema = the integration contract
③ Detective has zero write permissions. Ever.
④ Surgeon creates PRs only. No apply access. Ever.
⑤ AI never runs terraform apply. Full stop. Non-negotiable.

DevOps Conf 2026

DriftEvent: THE CONTRACT

$ cat code/core/schemas.py

```
@dataclass
class DriftEvent:
    # ── Core identity ──────────────────────────
    event_id:      str
    resource_id:   str  # "sg-0abc123"
    resource_type: str  # "aws_security_group"
    domain:        DriftDomain    # INFRASTRUCTURE | SECURITY | NETWORK
    severity:      DriftSeverity  # CRITICAL | HIGH | MEDIUM | LOW
    detected_at:   datetime

    # ── State delta ────────────────────────────
    desired_state: dict  # What Terraform says
    actual_state:  dict  # What AWS actually has
    diff:          dict  # The differences

    # ── Graph enrichment ───────────────────────
    blast_radius: list[str]  # Affected downstream resources
    risk_score:   float      # 0.0 to 1.0

    # ── Operational context ────────────────────
    actor:             Optional[str]  # Who made the change (CloudTrail)
    policy_violations: list[str]      # OPA policy violations
    tags:              dict           # Environment, owner, etc.
    correlation_id:    Optional[str]  # Links related alerts
    active_incident:   bool           # THE HOTFIX GUARD

# DATA SOURCES:
# desired_state  ← Terraform .tfstate (S3 / TFC backend)
# actual_state   ← Live cloud API (boto3 / AWS SDK)
# blast_radius   ← graph.get_blast_radius() output
# actor          ← CloudTrail lookup (WHO did this?)
# active_incident ← PagerDuty/OpsGenie webhook
#   - Effect: blocks ALL autonomous remediation
#   - Without this: you get the hotfix reversal story
```


DevOps Conf 2026

7 ALERTS → 1 INCIDENT

$ ./correlate.py --verbose

[BEFORE: 7 alerts, 3 tools, 3 teams]

```
14:23  [TERRAFORM]  sg-0abc123 — cidr_blocks changed
14:23  [TERRAFORM]  sg-0abc123 — description changed
14:24  [OPA]        no_public_ssh_access VIOLATED
14:24  [OPA]        sg_description_required VIOLATED
17:44  [CSPM]       Port 22 open to internet
17:44  [CSPM]       SSH unrestricted access
09:15  [NETOPS]     VPC flow anomaly (sg-0abc123)

→ 7 alerts · 3 teams · 3 queues · 19-hour exposure
```

[AFTER: 1 incident, full context]

```
14:23  [UNIFIED]  INC-20240315-042
  Resource:   sg-0abc123
(aws_security_group.api_server_sg)
  Severity:   CRITICAL  |  Risk: 0.87
  Domain:     SECURITY  |  Actor: john.smith (CloudTrail)
  Violations: CIS §5.2, sg_description_required
  Blast:      4 downstream resources

→ 1 event · correct team · full context · 5-min response
```

Correlation Engine Working

```python
WINDOW_SECONDS = 900  # 15-minute correlation window

def correlate(events: list[DriftEvent]) -> list[DriftEvent]:
    groups = defaultdict(list)
    for event in events:
        bucket = int(event.detected_at.timestamp()) // WINDOW_SECONDS
        groups[(event.resource_id, bucket)].append(event)
    return [merge_group(g) for g in groups.values()]

def merge_group(events: list[DriftEvent]) -> DriftEvent:
    primary = max(events, key=lambda e: ORDER[e.severity])  # Highest severity
    primary.correlation_id = str(uuid.uuid4())[:8]
    primary.policy_violations = list(set(v for e in events for v in e.policy_violations))
    return primary
```

~85% reduction in alert volume. 100% retention of critical signal. Trust in the system → team responds to alerts.

DevOps Conf 2026

THREE-AGENT ARCHITECTURE

```
$ ls -la ./agents/
```

[DETECTIVE]

Role: **Root Cause Analysis**

Tools (read-only):
- CloudTrail
- OPA evaluation
- Graph blast radius

Output:
→ RCAReport
→ auto_remediable

Permissions:
READ ONLY

[SURGEON (MAKER)]

Role: **Generate Patch**

Actions:
- Generate Terraform
- Self-correct (3x)

Output:
→ RemediationPlan
→ HCL patch

Permissions:
NONE (code only)

[CHECKER]

Role: **Validate**

3-Stage:
1. Syntax
2. Destructive
3. OPA policy

Output:
→ CheckerReport
→ approve/reject

Rejects: 25%

[MAKER-CHECKER PATTERN: MAKER generates · CHECKER validates · HUMANS approve]

```
[1] Drift detected → Detective analyzes → RCAReport
[2] If auto_remediable: Surgeon (MAKER) generates patch
[3] Checker (CHECKER) validates patch
    Stage 1: terraform validate (syntax)
    Stage 2: Destructive ops detection (semantic)
    Stage 3: OPA policy check (compliance)
[4] If validation passes: Create PR with full context
[5] Human review → OPA gate → Merge → Apply

SAFETY: All agents have zero AWS write permissions.
        Only GitHub PR creation (after Checker approval).
        No terraform apply. Maker cannot approve own work.
```


DevOps Conf 2026

DETECTIVE: IMPLEMENTATION

$ cat code/agents/detective_agent.py

```
class DetectiveAgent:
    """Read-only analyst. Zero write access."""

    def analyze(self, event: DriftEvent) -> RCAReport:
        # Step 1: Who made this change? (CloudTrail lookup)
        actor = self._query_cloudtrail(
            resource_id=event.resource_id,
            time_window=event.detected_at,
            actions=RESOURCE_ACTION_MAP[event.resource_type]  # ← KEY OPTIMIZATION
        )

        # Step 2: What policy does this violate? (OPA eval)
        violations = self.opa.evaluate(
            policy="drift/violations",
            input={
                "resource_type":  event.resource_type,
                "actual_state":   event.actual_state,
                "desired_state":  event.desired_state,
                "environment":    event.tags.get("environment"),
            }
        )

        # Step 3: LLM synthesis → structured RCAReport output
        report: RCAReport = self.llm.complete(
            prompt=self._build_rca_prompt(event, actor, violations),
            response_format=RCAReport  # ← Pydantic schema, not free text
        )
        return report
```

```
# RESOURCE_ACTION_MAP: The Performance Key

RESOURCE_ACTION_MAP = {
    "aws_security_group": [
        "AuthorizeSecurityGroupIngress",
        "RevokeSecurityGroupIngress",
        "ModifySecurityGroupRules",
    ],
    "aws_iam_role": ["AttachRolePolicy", "PutRolePolicy",
"UpdateAssumeRolePolicy"],
    "aws_route_table": ["CreateRoute", "DeleteRoute",
"ReplaceRoute"],
}

Without it: scan ALL CloudTrail events → expensive, slow
With it: look for SPECIFIC mutation events → fast, targeted
Build this map incrementally per resource type. Worth the
maintenance overhead.
```


DevOps Conf 2026

DETECTIVE OUTPUT: COMPLETE RCA


```
$ drift-cli analyze --event-id evt_sg_001
```

```
[DRIFT EVENT] evt_sg_20240315_001
  Resource:  sg-0abc123  (aws_security_group.api_server_sg)
  Domain:    SECURITY  ·  Severity: CRITICAL
  Detected:  2024-03-15  14:23:11 UTC

[DELTA] (what changed)
  ingress[0].cidr_blocks : ["10.0.0.0/8"] → ["0.0.0.0/0"]
  ingress[0].description : "Internal SSH only" → ""

[ACTOR] (CloudTrail LookupEvents)
  User:      arn:aws:iam::123456789012:user/john.smith
  Action:    AuthorizeSecurityGroupIngress
  Time:      2024-03-15 14:19:44 UTC  (4 min before detection)
  Source IP: 203.0.113.42  ← external IP, not in VPN range
  Via:       AWS Management Console

[BLAST RADIUS] (graph, depth=3)
  ├─ i-0a1b2c3d  EC2:app-prod-1      direct SSH exposure
  ├─ i-0e5f6g7h  EC2:app-prod-2      direct SSH exposure
  ├─ db-prod     RDS:primary         via exposed EC2
  └─ fs-0a1b2c   EFS:mount           on exposed instances

[POLICY VIOLATIONS]
  [CRITICAL]  CIS AWS §5.2  —  no_public_ssh_access
  [HIGH]      Internal policy  —  sg_description_required

[RISK SCORE]  0.96 / 1.0

[ROOT CAUSE]
  Summary:    Developer opened SSH to internet for debugging.
              Change not reverted.
  Root Cause: john.smith modified sg-0abc123 at 14:19 via Console
              from external IP (probable remote session)
  Intent:     Temporary debug access. High confidence:
              external IP · ingress-only change · no ticket found
  Auto-Remediable: FALSE
  Reason: security domain + production + risk 0.87
          Requires label: secops/approved on remediation PR

[OK] Forwarding RCAReport to Surgeon Agent...
```

```
Base criticality: aws_security_group = 0.8
Blast radius: 4 resources
Multiplier: 1.0 + (4 × 0.05) = 1.2
Score: 0.8 × 1.2 = 0.96 (capped at 1.0)
→ 0.96
```


DevOps Conf 2026

MAKER-CHECKER: WHY 2 AGENTS?

```
$ cat architecture.md | grep -A5 'Maker-Checker'
```

[MAKER: SURGEON AGENT]

```
ROLE: Generate remediation patches

OPTIMIZED FOR:
  [OK] Speed - generate patch in 2-3s
  [OK] Creativity - find right fix
  [OK] Context - understand RCA

CONSTRAINTS:
  - Temperature: 0.0 (deterministic)
  - Structured output (Pydantic)
  - ~2,600-word prompt template
  - Self-correction loop (3 retries)

OUTPUT:
  RemediationPlan:
    - terraform_patch: str
    - reasoning: str
    - affected_resources: list
    - risk_assessment: dict
```

[CHECKER: VALIDATION AGENT]

```
ROLE: Validate Surgeon's output

OPTIMIZED FOR:
  [OK] Safety - catch destructive ops
  [OK] Compliance - OPA policy checks
  [OK] Correctness - syntax validation

CONSTRAINTS:
  - Zero trust in Surgeon output
  - 3-stage validation pipeline
  - Conservative bias (reject > approve)
  - Separate model instance

OUTPUT:
  CheckerReport:
    - validation_passed: bool
    - destructive_changes: list
    - policy_violations: list
    - recommendation: str
```

[WHY SEPERATE AGENT??]

```
1. DIFFERENT OBJECTIVES
   Maker: "How do I fix this?" (generative, creative)
   Checker: "Is this fix safe?" (analytical, conservative)

2. PREVENT SELF-VALIDATION BIAS
   LLM validating its own output → 60% hallucination rate
   Separate agent → 95% accuracy (measured in production)

3. COMPLIANCE REQUIREMENT
   Financial services regulation: maker-checker for all infra changes
   Agent = Maker, Checker Agent = independent validator

4. STATISTICAL INDEPENDENCE
   Separate model instances, separate prompts, separate context
   Reduces correlated errors
```

CHECKER AGENT: 3-STAGE VALIDATION

$ cat code/agents/checker_agent.py

```
# 3-Stage Validation Pipeline (Maker-Checker Pattern)

class CheckerAgent:
    """Independent validator - zero trust on Surgeon output"""

    def validate(self, plan: RemediationPlan, drift: DriftEvent):

        # STAGE 1: SYNTAX (terraform fmt, < 1s)
        if not terraform_validate(plan.patch):
            return REJECT("Invalid HCL syntax")

        # STAGE 2: SEMANTIC (LLM-powered safety check)
        forbidden = ["force_destroy", "prevent_destroy=false",
                     "CIDR: 0.0.0.0/0", "data resource deletion"]
        if llm.detect_destructive_ops(plan.patch, forbidden):
            return REJECT("Destructive operation detected")

        # STAGE 3: POLICY (OPA evaluation against tier rules)
        if not opa.evaluate(plan.patch, drift.environment, drift.risk_score):
            return REJECT("Policy violation - tier threshold exceeded")

        return APPROVE()

# FAIL-FAST PIPELINE:
# [Syntax] ---> [Semantic] ---> [Policy] ---> APPROVE
#    |             |               |
#  REJECT        REJECT          REJECT

# PRODUCTION METRICS (30 days):
# Stage 1 rejects: 12% (syntax)  Stage 2: 8% (destructive)  Stage 3: 5% (policy)
# Total approval rate: 75%  |  Avg validation time: 2.3s

[FILE: code/agents/checker_agent.py] | [USES: ValidationPipeline, OPA]
```


DevOps Conf 2026

SURGEON: SELF-CORRECTION

$ cat code/agents/surgeon_agent.py

```
# PSEUDOCODE: Self-Correction Loop (Maker in Maker-Checker)

MAX_ATTEMPTS = 3

for attempt in range(MAX_ATTEMPTS):

    # 1. GENERATE (LLM with structured output)
    patch = llm.generate(
        prompt=build_prompt(rca, previous_errors),
        response_format=TerraformPatch,  # Pydantic schema enforced
        model="claude-sonnet-4.6"
    )

    # 2. VALIDATE (three-stage pipeline)

    # Stage 1: Syntax
    if terraform_validate(patch).has_errors:
        previous_errors = result.errors
        continue  # Retry with error context

    # Stage 2: Destructive check
    if terraform_plan(patch).has_destructive_changes:
        escalate_to_human("Destructive operation detected")
        return None

    # Stage 3: OPA policy
    if not opa.evaluate(patch).allowed:
        previous_errors = policy.violations
        continue  # Retry with policy feedback

    # SUCCESS: Create PR and exit
    return create_pr(patch, rca, validation_results)

# All attempts exhausted → human escalation
escalate_to_human(rca, all_errors)

# PRODUCTION STATS: 75% first-try | 95% after 3 attempts | 5% escalated
[FILE: code/agents/surgeon_agent.py]  [RETRY LOOP]
```


DevOps Conf 2026

DOMAIN-SPECIFIC PROMPTS

$ cat prompt_templates.md

[1] INFRASTRUCTURE DOMAIN

```
ROLE: You are a Senior Platform Engineer specializing in Terraform IaC.

CONTEXT:
  Resource:    {{resource_type}}     (e.g., aws_security_group)
  Drifted:     {{fields}}            (e.g., cidr_blocks, description)
  Desired:     {{desired_value}}     (from Terraform state)
  Environment: {{environment}}       (prod/staging/dev)

TASK: Generate ONLY the minimal HCL patch to restore desired state.

CONSTRAINTS:
  - Modify ONLY the drifted attributes
  - Preserve all other existing configurations
  - Use exact syntax from Terraform {{provider_version}}

OUTPUT FORMAT: Raw HCL block. No markdown. No explanations.
```

[2] SECURITY DOMAIN

```
ROLE: You are a Senior Security Engineer. CIS AWS Benchmark specialist.

CONTEXT:
  Violations:      {{policy_violations}}  (e.g., CIS 5.2: SSH open to internet)
  Blast Radius:    {{affected_resources}} (downstream EC2, RDS, EFS)
  Risk Score:      {{risk_score}}         (0.0 - 1.0)
  Actor:           {{cloudtrail_user}}    (who made the change)

TASK: Generate minimal security fix addressing ONLY violations.

HARD CONSTRAINTS:
  [NEVER] Broaden CIDR ranges (security regression)
  [NEVER] Open additional ports
  [NEVER] Weaken encryption settings
  [ALWAYS] Preserve compliant rules unchanged

OUTPUT: HCL patch + brief security justification (2 lines max).
```

[3] NETWORK DOMAIN

```
ROLE: You are a Senior Network Engineer. AWS VPC connectivity
specialist.

CONTEXT:
  Changed:     {{network_component}}  (route_table, NACL, peering)
  Traffic:     {{affected_services}}  (services using this route)
  Flow Logs:   {{anomaly_detected}}   (yes/no)

TASK: Restore desired routing state with traffic-awareness.

SAFETY CONSTRAINTS:
  - If route serves ACTIVE production traffic → FLAG for human review
  - Never silently drop routes for critical services
  - Validate peering/VPN connectivity before route changes

OUTPUT: HCL patch + traffic impact assessment + rollback plan.
```


DevOps Conf 2026

THE SELF-HEALING PR


```
[PULL REQUEST] #1337: [DRIFT:CRITICAL] Revert sg-0abc123 SSH exposure
drift-remediation-bot opened 8 minutes ago  ·  [AWAITING] secops/approved

[RISK ASSESSMENT]
  Severity:   CRITICAL
  Risk Score: 0.87 / 1.0
  Tier:       Tier 3 (requires security approval)
  Environment: production

[ROOT CAUSE ANALYSIS]
  Actor:      john.smith (arn:aws:iam::123456789012:user/john.smith)
  Action:     AuthorizeSecurityGroupIngress via AWS Console
  Timestamp:  2024-03-15 14:19:44 UTC (external IP: 203.0.113.42)
  Intent:     Temporary debug access (no ticket found)
  Reasoning:  Developer opened SSH to internet for debugging. Not reverted.

[BLAST RADIUS]
  Direct Impact:   2 EC2 instances (i-0a1b2c3d, i-0e5f6g7h)
  Indirect Impact: 1 RDS primary, 1 EFS mount (reachable via exposed EC2)
  Total Resources: 4 at risk

[PROPOSED FIX]
  Files Changed: 1 file  +2 -2 lines

  --- a/terraform/security_groups.tf
  +++ b/terraform/security_groups.tf
  @@ -12,8 +12,8 @@ resource "aws_security_group" "api_server_sg" {
     ingress {
       from_port   = 22
       to_port     = 22
  -    cidr_blocks = ["0.0.0.0/0"]
  -    description = ""
  +    cidr_blocks = ["10.0.0.0/8"]
  +    description = "Internal SSH only"
     }
   }

[VALIDATION RESULTS]
  [OK]   terraform validate   PASSED
  [OK]   terraform plan       PASSED (0 destructive changes)
  [OK]   Checker Agent        APPROVED (no destructive operations)
  [WAIT] OPA policy gate      BLOCKED (requires secops/approved label)

[POLICY VIOLATIONS FIXED]
  - CIS AWS Benchmark §5.2: SSH (port 22) should not be open to 0.0.0.0/0
  - Internal Policy: Security groups must have descriptions

[REQUIRED ACTIONS]
  [ ] SecOps: Add label 'secops/approved' to unblock CI/CD
  [ ] Confirm john.smith debug session is complete
  [ ] Consider creating bastion host for future debug access

[METADATA]
  Circuit Breaker: 1/3 attempts used
  Generated By:    drift-agent v1.2.0 (Azure o3-mini)
  CODEOWNERS:      @org/secops-team (auto-assigned)
  Auto-merge:      DISABLED (security domain + prod + risk > 0.3)
```

```
$ gh pr view 1337
```

THE REAL FEAR + THE REFRAME

"Will the AI delete my database?"

[THE FEARS (All Valid)]

```
[X]  Agent reverts a hotfix mid-incident
[X]  Agent generates a cascade-triggering patch
[X]  Agent auto-merges something subtly destructive
[X]  Agent confused by complex state dependencies
[X]  Agent outpaces a human's ability to react
[X]  Agent creates compliance violation while fixing
one
[X]  Agent manipulated via prompt injection through
     resource names or tag values
```

[THE REFRAME]

```
TREAT THE AGENT AS A MACHINE IDENTITY
Not magic. Not autonomous. Not trusted by default.
A service account that writes code.

Apply the same controls as any CI/CD bot:

IAM — AGENT SERVICE ROLE
  cloudtrail:LookupEvents           [OK]
  s3:GetObject (state bucket)       [OK]
  opa:Evaluate                      [OK]
  github:CreatePullRequest          [OK]
  ──────────────────────────────────────
  ec2:AuthorizeSecurityGroupIngress [DENY] NEVER
  iam:AttachRolePolicy              [DENY] NEVER
  Any resource mutation API         [DENY] NEVER
  terraform apply                   [DENY] NEVER
```

```
[THE QUESTION SHIFT]

[WRONG]  "Can I trust the AI?"

[RIGHT]  "Have I governed this service account correctly?"

That's an engineering problem. We know how to solve it.

Every action: logged, attributable, auditable.
All changes: through the same GitOps pipeline as any human engineer.

Once you accept this framing, governance becomes a familiar problem:
what are the right constraints for this service account,
and how do I enforce them declaratively? OPA is the answer.
```


DevOps Conf 2026

THE REAL FEAR + THE REFRAME

"Will the AI delete my database?"

[THE FEARS (All Valid)]

- [X] Agent reverts a hotfix mid-incident
- [X] Agent generates a cascade-triggering patch
- [X] Agent auto-merges something subtly destructive
- [X] Agent confused by complex state dependencies
- [X] Agent outpaces a human's ability to react
- [X] Agent creates compliance violation while fixing one
- [X] Agent manipulated via prompt injection through resource names or tag values

[THE REFRAME]

Fear 1: Reverts hotfix mid-incident
Solution: active_incident flag blocks remediation
Engineering control, not trust

Fear 2: Cascade-triggering patch
Solution: terraform plan shows destructive changes
Validation blocks it

Fear 3: Auto-merges destructive change
Solution: OPA policies gate auto-merge
Critical resources require review

Fear 4: Confused by complex state
Solution: Validation loop catches errors
Escalates to human after 3 attempts

Fear 5: Outpaces human reaction
Solution: Circuit breaker limits rate
Max 3 remediations per hour per resource

Fear 6: Creates new compliance violation
Solution: OPA evaluates against all policies
Multi-policy validation

Fear 7: Prompt injection
Solution: Structured data input
Constrained output
Validation catches malformed code

Every fear has an engineering mitigation.

"Once you accept this framing, governance becomes a familiar problem: what are the right constraints for this service account, and how do I enforce them declaratively? OPA is the answer."

DevOps Conf 2026

OPA GUARDRAILS: 3-TIER


```
# TIER 1: AUTO-MERGE (~40% volume)  [code/policies/tier1.rego]

default auto_approve = false

auto_approve {
    input.risk_score < 0.3                          # Threshold
    not critical_resource_types[input.resource_type]
    not input.active_incident  # ← THE HOTFIX GUARD
    tag_only_change
}

# TIER 2 & 3: DENY RULES (Hard blocks)

deny[msg] {
    input.active_incident == true
    not input.pr_labels["incident/override"]
    msg := "BLOCK: Active incident. Prevents hotfix reversal."
}

deny[msg] {
    input.has_destructive_changes == true
    msg := "BLOCK: Destructive changes never auto-approved."
}

deny[msg] {
    critical_resource_types[input.resource_type]
    not input.pr_labels["drift/approved"]
    msg := sprintf("BLOCK: '%v' requires approval", [input.resource_type])
}

deny[msg] {
    input.domain == "security"
    not input.pr_labels["secops/approved"]
    msg := "BLOCK: Security domain requires SecOps."
}

# HELPERS

critical_resource_types := {
    "aws_db_instance", "aws_iam_role", "aws_kms_key",
    "aws_vpc", "aws_secretsmanager_secret"
}

tag_only_change {
    changed_keys := {k | _ = input.diff[k]}
    changed_keys == {"tags"}
}
```

```
$ cat code/policies/*.rego
# 3 tiers combined
```


DevOps Conf 2026

WHY active_incident EXISTS

Having the field in the schema ≠ Having the integration to populate

```
PRODUCTION INCIDENT: Hotfix Reversal
──────────────────────────────────────────────────

[14:02] [DEGRADED] Service P99 latency: 8s (threshold: 200ms)
[14:07] [PAGED]    SRE anna.kim receives P1 alert
[14:11] [HOTFIX]   Root cause: RDS connection timeout
                   Anna applies fix via Console (no time for PR)
                   Modified: db-prod security group sg-0abc123
[14:14] [RESOLVED] Service healthy - latency back to 95ms

──────── THEN AUTOMATION FIRES ────────────────

[14:19] [DRIFT]    Detection: anna's Console change detected
                   active_incident: NOT SET (no PagerDuty integration yet)
                   OPA risk: 0.45 (below 0.5 auto-merge threshold for instance)
                   Agent generated patch: reverts anna's fix
                   Atlantis auto-merged | terraform apply executed

[14:23] [DEGRADED] SERVICE DEGRADES AGAIN
                   Anna unaware fix was reverted
                   Thinks: "I misdiagnosed the problem"
                   Re-investigates from scratch
──────────────────────────────────────────────────

[IMPACT] 22 ADDITIONAL MINUTES OF P1 DOWNTIME

LESSON: The agent was technically correct.
        The SYSTEM was wrong.
        Technically correct + operationally blind = P1 extension.

FIXES IMPLEMENTED:
[OK] PagerDuty → EventBridge → sets active_incident flag
[OK] OPA: active_incident now blocks ALL remediation attempts
[OK] Auto-merge threshold lowered: 0.6 → 0.3 (require more reviews)
[OK] incident/override label added as escape hatch
[OK] All Console changes: 2-hour suppression window
```


DevOps Conf 2026

CROSS-TEAM APPROVAL WORKFLOW


```
[THE FULL FLOW]

1. Surgeon creates PR + sets domain label
2. GitHub CI validates (terraform validate, plan, OPA)
3. Labels set by OPA result (auto-merge / review-required / blocked)
4. CODEOWNERS routes to correct team
5. AUTO-MERGE PATH (if risk < 0.3 in dev):
   → CI runs terraform apply
   → Post-apply drift check
   → Either resolved OR circuit breaker increments

6. OR HUMAN REVIEW PATH (if risk ≥ 0.3 or prod):
   → Team adds approval label
   → CI runs terraform plan (again)
   → Human merges PR
   → CI runs terraform apply
   → Post-apply verification

[THE REJECTION PATH — CRITICAL FOR LEARNING]

When PR closed without merge:
  1. Parse human comment: "intentional deviation — reason"
  2. Add resource+attribute to suppression list
  3. Log decision to audit trail
  4. Future drift on same attribute suppressed

[POST-APPLY LOOP DETECTION]

After terraform apply, recheck drift:
  - If still drifted → increment circuit breaker
  - At 3 remediations → page human (loop detection)
  - Prevents infinite remediation loops

KEY INSIGHT:
You're not building a new approval system. You're using CODEOWNERS,
PR labels, branch protection rules, and Atlantis or GitHub Actions —
primitives you already have.
```


DevOps Conf 2026

CIRCUIT BREAKER: LOOP PREVENTION

$ cat code/core/circuit_breaker.py

```
[THE LOOP SCENARIO: Autoscaling vs Terraform]

T+0:00  Autoscaling: t3.micro → t3.large (load spike)
T+0:01  Drift detected (risk: 0.22 — low)
T+0:03  Surgeon reverts to t3.micro
T+0:04  Auto-merge, terraform apply
T+0:07  Load still high, autoscaling fires again
T+0:08  Drift detected again · Patch generated again
T+0:10  Auto-merge, apply again

→ 7 cycles in 1 hour
→ Instance thrashing constantly
→ No alert (each below threshold)
→ Nobody notices until bill arrives

Loops are the most dangerous failure mode — specifically
because they can be completely silent.
```

```
# PSEUDOCODE: Redis-Backed Circuit Breaker

class RemediationCircuitBreaker:
    WINDOW_SECONDS = 3600   # 1-hour rolling window
    MAX_REMEDIATIONS = 3    # before circuit trips
    COOLDOWN_SECONDS = 1800 # 30-min lockout

    def can_remediate(self, resource_id: str) -> tuple[bool, str]:
        # Check cooldown
        if self.redis.exists(f"circuit:cooldown:{resource_id}"):
            return (False, "Circuit open. Manual investigation required.")

        count = int(self.redis.get(f"circuit:count:{resource_id}") or 0)

        if count >= self.MAX_REMEDIATIONS:
            self.redis.setex(cooldown_key, COOLDOWN_SECONDS, "tripped")
            self._page_human(resource_id, count)  # PagerDuty (not Slack)
            return (False, f"Loop detected: {count} remediations")

        return (True, "ok")

WHY REDIS (not in-memory):
- Agent processes restart (Kubernetes pods)
- In-memory counters reset → loops not detected
- Redis persists across restarts
```


DevOps Conf 2026

ALERT FATIGUE: HOW WE BROKE IT


```
[FIRST TWO WEEKS]

Monday:    847 alerts
Tuesday:   1,103 alerts
Wednesday: 923 alerts
Thursday:  890 alerts

TEAM RESPONSE BY DAY 10:
[X]  Disabled Slack notifications
[X]  Filtered drift emails to archive
[X]  Systematically ignored all alerts

OUTCOME: Detection system so noisy the team developed a systematic
habit of ignoring it. This is worse than no detection system.
You have false confidence AND noise.
Real incidents drown in false positives.
```

```
[THE 5 FIXES]

1. Correlation: 7 alerts → 1 incident  (-70% volume immediately)

2. Severity-Based Routing:
   CRITICAL → PagerDuty page  |  HIGH → Slack alert
   MEDIUM → hourly batch      |  LOW → daily digest

3. Drift-Ignore Annotations:
   # drift-ignore: instance_type
   # Reason: managed by autoscaling policy

4. Per-Attribute Cooldowns:
   First: alert  |  Second: alert with "repeat"  |  Third+: daily digest

5. Rejection Feedback Loop:
   PR closed + human comment → attribute added to suppression list
```

```
[THE RESULT]

Week 1:  ~900 alerts/day  (team ignores everything)
Week 4:   ~30 alerts/day  (team responds to everything)

The reduction is the outcome.
Team trust in the signal is the actual product.
```


DevOps Conf 2026

HONEST LIMITS: RISK MATRIX

$ cat production_checklist.txt

```
[RISK MATRIX: When Automation Is Appropriate]

                    RESOURCE CRITICALITY
                 LOW                    HIGH
          ┌──────────────────────┬──────────────────────┐
DEV /     │  [AUTO] FULL AUTO    │  [REVIEW] REQUIRED   │
STAGING   │  Auto-merge OK       │  PR + team review    │
          ├──────────────────────┼──────────────────────┤
PROD      │  [REVIEW] REQUIRED   │  [MANUAL] ONLY       │
          │  PR required         │  Mandatory human     │
          └──────────────────────┴──────────────────────┘

Examples:
  [AUTO]   Tag drift in dev · non-critical attrs in staging
  [REVIEW] Instance type in prod · S3 lifecycle in prod
  [MANUAL] RDS, IAM, KMS, VPC, Secrets Manager — ANY environment
```

```
[MANDATORY PRE-PRODUCTION CHECKLIST]

Before enabling ANY auto-remediation:

[OK] 1.  State locking active (S3+DynamoDB or TFC)
[OK] 2.  OPA guardrails deployed AND tested with real deny scenarios
[OK] 3.  Incident integration live (PD/OpsGenie → active_incident)
[OK] 4.  Circuit breaker active with Redis backend
[OK] 5.  Audit logging for every agent action
[OK] 6.  Post-apply drift verification enabled
[OK] 7.  Rollback runbook: restore prior state in < 5 minutes
[OK] 8.  Alert thresholds tuned (< 50 actionable alerts/day)
[OK] 9.  Auto-merge scoped ONLY to risk < 0.3 + non-critical
[OK] 10. Kill switch: disable entire agent in < 2 minutes

If ANY item missing:
  - Detection only — YES
  - Detective agent (read-only analysis) — YES
  - Surgeon PR generation — YES
  - Auto-merge — NOT YET

KEY INSIGHT: Each item is there because its absence caused
a real incident somewhere. This is a lessons-learned list.
```


DevOps Conf 2026

STRUCTURED PROMPTS: THE SECRET

```
# WHY 95% ACCURACY? THE PROMPT TEMPLATE

FORBIDDEN PATTERNS (will cause Checker to reject):
[FORBIDDEN] provider "aws" { ... }        # No provider blocks
[FORBIDDEN] data "aws_*" { ... }          # No data sources
[FORBIDDEN] provisioner "local-exec"      # No provisioners
[FORBIDDEN] force_destroy = true          # No destructive flags
[FORBIDDEN] lifecycle { prevent_destroy = false }

ALLOWED PATTERNS:
[ALLOWED] resource "aws_security_group" "name" { ... }
[ALLOWED] ingress { cidr_blocks = ["10.0.0.0/8"] }
[ALLOWED] tags = { Environment = "prod" }

VALIDATION CHECKLIST (LLM must confirm):
[ ] Output is ONLY the resource block (no explanation text)
[ ] cidr_blocks matches desired_state exactly
[ ] No markdown code blocks (hcl format)
[ ] Resource type = {drift_event.resource_type}
[ ] Resource name = {drift_event.resource_name}
[ ] All modified fields present in output
[ ] No fields modified that weren't in drift

SECURITY CONSTRAINTS:
- If CIDR change: NEW cidr must be NARROWER or EQUAL (never broader)
- If port change: NEW port must be more restrictive
- If IAM change: NEW permissions must be subset of old
- ANY destructive operation → ERROR (will be rejected)

EXAMPLE OUTPUT (EXACTLY THIS FORMAT):
resource "aws_security_group" "api_server_sg" {
  ingress {
    from_port   = 22
    to_port     = 22
    cidr_blocks = ["10.0.0.0/8"]
    description = "Internal SSH only"
  }
}

COMMON ERRORS TO AVOID:
[X] Wrapping in markdown: hcl code blocks
[X] Adding explanation: "Here's the fix..."
[X] Including provider blocks
[X] Modifying unrelated fields
[X] Broadening CIDR (e.g., 10.0.0.0/8 → 0.0.0.0/0)

INPUT CONTEXT PROVIDED:
- desired_state: {rca_report.desired_state}
- actual_state: {drift_event.actual_state}
- policy_violations: {rca_report.policy_violations}
- blast_radius: {rca_report.blast_radius}

YOUR TASK:
Generate ONLY the Terraform resource block to restore desired_state.
No additional text. No explanation. Pure HCL.
```


DevOps Conf 2026

CAUSAL GRAPH: ROOT CAUSE AI

```
$ python -m core.causal_graph  # code/core/causal_graph.py
```

```
# PSEUDOCODE: Causal DAG Builder

def build_causal_graph(events):
    # Step 1: Create directed graph
    graph = DirectedGraph()

    # Step 2: Add all drift events as nodes
    for event in events:
        graph.add_node(event)

    # Step 3: Sort by time (cause precedes effect)
    sorted_by_time = sort(events, by=timestamp)

    # Step 4: Add causal edges
    for each (earlier_event, later_event) in
sorted_by_time:
        time_diff = later_event.time - earlier_event.time

        if time_diff < 1 hour:  # Temporal window
            # Look up domain relationship strength
            strength = CAUSAL_MATRIX[
                earlier_event.resource_type,
                later_event.resource_type
            ]

            if strength > 0:
                graph.add_edge(earlier → later,
weight=strength)

    return graph

# CAUSAL RELATIONSHIP MATRIX (domain knowledge):
security_group → ec2_instance:  0.9
iam_role       → lambda:        0.9
vpc            → subnet:         0.95
subnet         → ec2_instance:  0.9

# ROOT CAUSE = nodes with:
#   - Zero incoming edges (not caused by anything)
#   - At least 1 outgoing edge (causes something else)

root_causes = nodes where in_degree == 0 AND out_degree > 0
```

```
[CAUSAL GRAPH VISUALIZATION]

    sg-0abc123 (CRITICAL)
    aws_security_group
         |
         |  causes (0.9)
         v
    i-0a1b2c3d (HIGH)
    aws_instance
         |
         |  causes (0.7)
         v
    db-prod (MEDIUM)
    rds_instance

ROOT CAUSE: sg-0abc123
  - No incoming edges (nothing caused it)
  - 2 outgoing edges (caused 2 events)
  - Impact score: 0.87

DOWNSTREAM EFFECTS:
  - 2 EC2 instances (direct)
  - 1 RDS database (indirect, via EC2)
  - 1 EFS mount (indirect, via EC2)

CAUSAL DEPTH: 2 hops
  sg → instance → database

PREDICTION:
  "Fixing sg-0abc123 will resolve
   all 3 downstream alerts"

VISUALIZATION SAVED:
  demo/causal_graph.png

STATISTICS:
  Total nodes: 7 events
  Causal edges: 12 relationships
  Root causes: 1 identified
  Noise reduction: 85%
  (7 alerts → 1 root cause)
```


DevOps Conf 2026

TAKEAWAYS & RESOURCES

[KEY TAKEAWAYS]

1. Drift is inevitable → Correlation is essential
2. Graph model + Causal AI → Blast radius + RCA
3. Three-agent architecture → Detective (RCA) + Surgeon (MAKER) + Checker (VALIDATOR)
4. OPA + CODEOWNERS → Governance without gatekeeping
5. active_incident field → Prevent hotfix reversal
6. Start small → Scale with confidence
7. Circuit breakers → Prevent infinite loops
8. 85% alert reduction → Team trust

[RESOURCES]

- GitHub: https://github.com/neerajp99/devopsconfRU
  /schemas - DriftEvent + RCAReport (Pydantic)
  /agents - Detective + Surgeon implementations
  /opa - Three-tier policies (Rego)
  /prompts - Domain-specific templates

- Models: Claude Sonnet 4.6, Azure o3-mini, Claude Opus 4.6
- Tools: Neo4j, OPA, NetworkX, CausalLearn (PyWhy)
- Cost: <$300/month for 50 events/day - [Approx cost for deploying and running the agents]

- Contact: neeraj@lyntcube.com

For slides & repository

QUESTIONS?

> Thank you!

DevOps Conf 2026

Thank You

Neeraj Pandey
@n4jp4y

DevOps Conf 2026